

Apollo 对比方案

面向 PyTorch Discrete Choice Model 库的 Benchmark 设计

目标：将 Apollo 作为外部 reference implementation，系统比较新库在模型覆盖、估计数值一致性、统计推断、post-estimation、runtime 和 memory 上的表现。

Baichuan Mo
2026-07-04

计划对象	基于 PyTorch 的 discrete choice model estimation library
对标软件	Apollo R package, Biogeme, xlogit, torch-choice, statsmodels 风格 inference
核心产出	可复现 benchmark pipeline、JSON 规格文件、Apollo R backend、Python 比较脚本、结果表格

Contents

1	结论先行	3
2	Apollo 与 Python 的关系	3
2.1	是否有 Apollo 的官方 Python 版?	3
2.2	三种 Python 调用 Apollo 的方法	3
3	Benchmark 总体架构	4
3.1	核心 pipeline	4
3.2	建议目录结构	5
4	共享模型 specification	6
4.1	为什么需要中间 JSON?	6
4.2	MNL JSON 规格示例	6
4.3	Mixed logit JSON 规格示例	7
5	Apollo R backend 设计	7
5.1	Rscript 接口	7
5.2	Apollo 结果 JSON	8
6	五层对比指标	8
6.1	第一层: 模型覆盖对比	8
6.2	第二层: 数值一致性	9
6.3	第三层: inference 对比	10
6.4	第四层: prediction 与 post-estimation	10
6.5	第五层: runtime 与 memory	11
7	模型对比顺序	11
7.1	Step 1: MNL	11
7.2	Step 2: Nested logit / Cross-nested logit	12
7.3	Step 3: Mixed logit	12
7.4	Step 4: Latent class	13
7.5	Step 5: Hybrid choice / ICLV	14
8	Python 端比较脚本	14
8.1	结果读取和参数对比	14
8.2	自动化断言	15
9	Benchmark 数据集	15
10	常见坑与处理原则	15
10.1	变量 scaling	15
10.2	ASC normalization	16
10.3	Availability	16
10.4	Panel likelihood	16
10.5	Random draws	16
10.6	Optimizer convergence	16

11 可交付 benchmark 工具	16
12 开发里程碑	17
13 论文或文档中的 benchmark 描述	17
14 最小可执行计划	18
15 参考来源	19

1 结论先行

Apollo 没有官方原生 Python 实现。官方主页和 CRAN 页面均将 Apollo 定义为用于 choice model estimation and application 的 R package。对你的 PyTorch 库来说, 最合理的做法不是把 Apollo 改写成 Python, 而是把 Apollo 作为外部 gold-standard reference backend: Python 负责生成数据和模型规格, Rscript 调用 Apollo 估计, Apollo 输出 JSON, Python 再逐项比较。

这份方案的核心判断是:

- Apollo 适合做 **reference implementation**, 不是你的 Python library dependency。
- 最推荐的调用方式是 **Python subprocess + Rscript**, 而不是一开始就用 rpy2 深度嵌入。
- 对比不能只比速度, 要分成五层: 模型覆盖、数值一致性、inference、post-estimation、runtime/memory。
- Mixed logit 和 ICLV 不应一开始就比较最终估计值。要先在固定参数 θ_0 下比较 likelihood 和 predicted probability。
- 你的库真正可能超越 Apollo 的地方是: PyTorch GPU、ragged choice-set kernel、panel mixed logit streaming likelihood、per-observation score 加速、statsmodels 风格 robust/cluster inference 和 sklearn 风格 API。

2 Apollo 与 Python 的关系

2.1 是否有 Apollo 的官方 Python 版?

截至本计划整理时, Apollo 的官方定位仍是 **R package**。官方主页描述其为 flexible R package, 支持用户自定义模型函数、random heterogeneity、standalone/hybrid model structures、classical/Bayesian estimation、多线程估计以及大量 pre/post-estimation routines。CRAN 页面也将其列为 R package, 并说明其支持 discrete choice、random heterogeneity、hybrid structures、Bayesian/classical estimation、discrete-continuous models 等功能。

因此, 和 Apollo 对比的正确工程方式是:

把 Apollo 当成一个由 Rscript 驱动的 reference solver, 而不是作为 Python 内部模块。

2.2 三种 Python 调用 Apollo 的方法

方法	做法	建议
subprocess + Rscript	Python 生成 CSV/JSON, 调用 Rscript run_model.R, Apollo 输出 JSON, Python 读取结果。	最推荐。 稳定、容易复现、适合 CI 和 benchmark, 不强制用户安装 R 依赖。

方法	做法	建议
rpyp2	Python 中导入 R package, 并在 Python 进程内执行 Apollo R 代码。	可行但调试复杂。Apollo 模型通常依赖 R 环境、全局变量和函数闭包, 不适合作为第一版主路线。
Docker	固定 R、Apollo、Python、PyTorch、BLAS/OpenMP、CUDA 版本。	适合软件论文和公开 benchmark。可作为后期 reproducibility layer。

推荐的 Python 调用方式如下:

```
import subprocess
import json
from pathlib import Path

cmd = [
    "Rscript",
    "benchmarks/apollo/R/run_mnl_swissmetro.R",
    "--data", "benchmarks/data/swissmetro.csv",
    "--spec", "benchmarks/specs/swissmetro_mnl.json",
    "--output", "benchmarks/results/apollo_mnl_swissmetro.json",
]

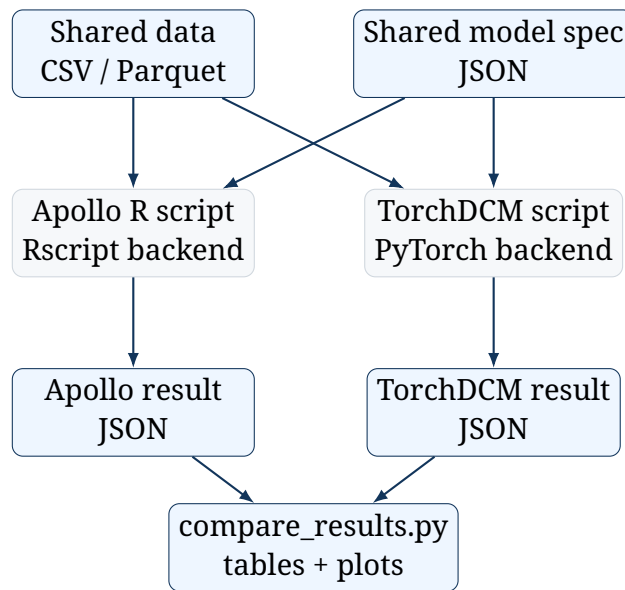
subprocess.run(cmd, check=True)

with open("benchmarks/results/apollo_mnl_swissmetro.json") as f:
    apollo_result = json.load(f)
```

3 Benchmark 总体架构

3.1 核心 pipeline

建议建立一个统一 benchmark pipeline, 让 Apollo 和你的 PyTorch 库共享同一份数据、模型 specification、初始值、固定参数、availability、random seed 和 draws 设置。



3.2 建议目录结构

```

benchmarks/
  data/
    swissmetro.csv
    apollo_mode_choice.csv
    synthetic_mnl_large.parquet
    synthetic_panel_mmnl.parquet
  specs/
    swissmetro_mnl.json
    swissmetro_nl.json
    apollo_mode_choice_mnl.json
    synthetic_mmnl_panel.json
  apollo/
    R/
      run_mnl.R
      run_nl.R
      run_cnl.R
      run_mixed_logit.R
      run_latent_class.R
      extract_apollo_result.R
      install_apollo.R
  torchdcm/
    run_mnl.py
    run_nl.py
    run_mixed_logit.py
    run_latent_class.py
  results/
    apollo_*.json
    torchdcm_*.json
    benchmark_report.csv
  compare_results.py
  
```

4 共享模型 specification

4.1 为什么需要中间 JSON?

如果 Apollo 和 PyTorch 库各自手写模型脚本，很容易出现以下不一致：

- ASC normalization 不一致；
- time/cost scaling 不一致；
- availability 处理不一致；
- fixed parameter 不一致；
- random draws 不一致；
- panel likelihood 分组不一致；
- optimizer tolerance 不一致。

因此要定义一份中间 JSON specification，再分别翻译成 Apollo R 代码和 TorchDCM Python 代码。

4.2 MNL JSON 规格示例

```
{
  "model": "mnl",
  "alternatives": ["car", "train", "sm"],
  "obs_id_col": "obs_id",
  "choice_col": "choice",
  "choice_format": "alternative_name",
  "availability": {
    "car": "av_car",
    "train": "av_train",
    "sm": "av_sm"
  },
  "utilities": {
    "car": "ASC_CAR + B_TIME * time_car + B_COST * cost_car",
    "train": "ASC_TRAIN + B_TIME * time_train + B_COST * cost_train",
    "sm": "B_TIME * time_sm + B_COST * cost_sm"
  },
  "parameters": {
    "ASC_CAR": 0.0,
    "ASC_TRAIN": 0.0,
    "B_TIME": -0.01,
    "B_COST": -0.10
  },
  "fixed": [],
  "base_alternative": "sm",
  "weights_col": null,
  "scale": {
    "time": "minutes",
    "cost": "currency_units"
  }
}
```

4.3 Mixed logit JSON 规格示例

```
{
  "model": "mixed_logit",
  "panel_id_col": "person_id",
  "alternatives": ["car", "train", "sm"],
  "obs_id_col": "obs_id",
  "choice_col": "choice",
  "utilities": {
    "car": "ASC_CAR + B_TIME * time_car + B_COST * cost_car",
    "train": "ASC_TRAIN + B_TIME * time_train + B_COST * cost_train",
    "sm": "B_TIME * time_sm + B_COST * cost_sm"
  },
  "parameters": {
    "ASC_CAR": 0.0,
    "ASC_TRAIN": 0.0,
    "MU_TIME": -3.0,
    "SIGMA_TIME": 0.5,
    "MU_COST": -0.1,
    "SIGMA_COST": 0.02
  },
  "random_parameters": {
    "B_TIME": {"dist": "lognormal", "sign": "negative", "mean": "MU_TIME", "sd": "SIGMA_TIME"},
    "B_COST": {"dist": "normal", "mean": "MU_COST", "sd": "SIGMA_COST"}
  },
  "draws": {
    "type": "sobol",
    "n_draws": 512,
    "seed": 20260704,
    "antithetic": true,
    "common_random_numbers": true
  }
}
```

5 Apollo R backend 设计

5.1 Rscript 接口

每个 Apollo 脚本都应该支持统一命令行参数:

```
Rscript benchmarks/apollo/R/run_mnl.R \
--data benchmarks/data/swissmetro.csv \
--spec benchmarks/specs/swissmetro_mnl.json \
--output benchmarks/results/apollo_mnl_swissmetro.json \
--threads 4
```

建议每个 R 脚本遵循同一结构:

1. 读取 JSON specification;
2. 读取 CSV/Parquet 数据;
3. 调用 `apollo_initialise()`;

4. 构造 `apollo_control`;
5. 构造 `apollo_beta` 和 `apollo_fixed`;
6. 构造 `apollo_probabilities`;
7. 调用 `apollo_estimate()`;
8. 调用统一的 `extract_apollo_result()`;
9. 输出 JSON。

5.2 Apollo 结果 JSON

Apollo 端不建议只保存文本 `summary`，而要输出机器可读 JSON：

```
library(apollo)
library(jsonlite)

# after model <- apollo_estimate(...)
out <- list(
  backend = "apollo",
  model = "mnl",
  apollo_version = as.character(packageVersion("apollo")),
  loglik = as.numeric(model$maximum),
  estimates = as.list(model$estimate),
  se = as.list(model$se),
  robust_se = as.list(model$robse),
  t_ratio = as.list(model$t.ratio),
  n_obs = nrow(database),
  convergence = list(
    status = model$code,
    message = model$message
  )
)

write_json(out, output_path, pretty = TRUE, auto_unbox = TRUE)
```

实际 Apollo 对象字段名可能因版本和模型而略有差异，所以应将字段抽取逻辑集中放在：

`benchmarks/apollo/R/extract_apollo_result.R`

6 五层对比指标

6.1 第一层：模型覆盖对比

模型/功能	Apollo	TorchDCM 路线	对比重点
MNL / conditional logit	支持	v0.1	参数、LL、SE、概率、market share
Nested logit	支持	v0.2	nest parameter、inclusive value、LL

模型/功能	Apollo	TorchDCM 路线	对比重点
Cross-nested logit	支持	v0.2/v0.3	allocation、nest scale、概率
Mixed logit	支持	v0.3	simulated LL、random coefficients、posterior
Latent class	支持	v0.4	class-specific 参数、membership、posterior
Hybrid choice / ICLV	支持	v0.5	latent variable、measurement equation、joint likelihood
MDCEV / discrete-continuous	支持	later	discrete-continuous likelihood
Bayesian estimation	支持	optional	posterior summary
Post-estimation	支持	v0.1+	WTP、elasticity、counterfactual、logsum

短期目标不是在模型覆盖上全面超过 Apollo，而是先在 MNL、NL、mixed logit 三类核心模型上做到数值一致，并展示 PyTorch/C++/CUDA 在大规模数据和 inference 计算上的优势。

6.2 第二层：数值一致性

对于每个模型，应比较以下核心数值：

$$\Delta_{\theta, \infty} = \left\| \hat{\theta}_{\text{torch}} - \hat{\theta}_{\text{Apollo}} \right\|_{\infty}, \quad (1)$$

$$\Delta_{\ell} = \left| \ell_{\text{torch}}(\hat{\theta}_{\text{torch}}) - \ell_{\text{Apollo}}(\hat{\theta}_{\text{Apollo}}) \right|. \quad (2)$$

推荐输出：

- parameter estimates;
- log-likelihood at convergence;
- log-likelihood at zero;
- AIC / BIC;
- rho-square / adjusted rho-square;
- predicted probabilities;
- market shares;
- elasticities;
- WTP / VOT。

6.3 第三层：inference 对比

你的库若要达到 statsmodels 级 inference，必须和 Apollo 对比以下统计输出：

Inference item	对比方式
Hessian-based SE	比较 classical standard error 和 covariance matrix
Robust SE	比较 sandwich covariance, 注明 finite-sample correction
Cluster SE / panel SE	对 repeated SP/RP 数据按 individual 聚类
t-stat / z-stat	比较 $\hat{\theta}/se$
p-value	比较显著性判断是否一致
confidence interval	比较 95% CI
Delta-method WTP	比较 WTP estimate、SE、CI
elasticity	比较 direct/cross elasticity

建议 covariance API 提供明确开关：

```
res.cov_params(cov_type="classic")
res.cov_params(cov_type="robust", correction=False)
res.cov_params(cov_type="robust", correction=True)
res.cov_params(cov_type="cluster", groups=data.person_id)
```

6.4 第四层：prediction 与 post-estimation

Apollo 不只是 estimation 工具，也有 forecasting 和 post-processing。因此 benchmark 需要比较：

预测概率

$$\max_{n,i} |P_{ni}^{\text{torch}} - P_{ni}^{\text{Apollo}}|. \quad (3)$$

Market share

$$S_i = \frac{1}{N} \sum_{n=1}^N P_{ni}. \quad (4)$$

Counterfactual simulation 示例包括：

- car cost 增加 10%；
- train time 降低 10%；
- 某个 alternative availability 从 0 变为 1；
- 新增 alternative；
- 改变 policy attribute，例如票价、发车频率、补贴。

WTP / VOT

$$\text{VOT} = -\frac{\beta_{\text{time}}}{\beta_{\text{cost}}}. \quad (5)$$

对比 VOT estimate、standard error 和 confidence interval。

6.5 第五层：runtime 与 memory

速度对比应该放在最后，而且要控制同样的模型和收敛标准。建议记录：

- number of observations N ;
- number of alternatives J ;
- number of parameters K ;
- number of simulation draws R ;
- panel length T ;
- CPU threads;
- GPU model;
- wall-clock time;
- peak memory;
- number of likelihood evaluations;
- number of gradient evaluations;
- final log-likelihood。

合理预期如下：

场景	预期
小规模 MNL	Apollo 已经很成熟，PyTorch 未必明显更快。
大规模 MNL + variable choice set	你的 ragged C++/CUDA logsumexp kernel 可能有优势。
Mixed logit + 大 draws	GPU 和 streaming logmeanexp kernel 是主要优势。
Panel mixed logit	fused panel likelihood kernel 是关键卖点。
Robust/cluster SE	vectorized AD 或 analytic score kernel 可能明显更快。

7 模型对比顺序**7.1 Step 1: MNL**

MNL 是第一个必须完成的 Apollo 对比任务。概率为：

$$P_{ni} = \frac{\exp(V_{ni})}{\sum_{j \in C_n} \exp(V_{nj})}. \quad (6)$$

对比项目：

- same utility specification;
- same ASC normalization;
- same variable scaling;
- same availability;
- same initial values;
- same log-likelihood;
- same gradient and Hessian at optimum;
- same standard errors;
- same predicted probabilities。

建议接受阈值：

```
abs(loglik_torch - loglik_apollo) < 1e-5
max_abs_param_diff < 1e-4
max_abs_probability_diff < 1e-6
max_abs_se_diff < 1e-4
```

7.2 Step 2: Nested logit / Cross-nested logit

Nested logit 的概率可写为：

$$P_{ni} = P_{ni|m} P_{nm}, \quad i \in m. \quad (7)$$

Inclusive value:

$$IV_{nm} = \log \sum_{j \in m} \exp \left(\frac{V_{nj}}{\lambda_m} \right). \quad (8)$$

Nest choice probability:

$$P_{nm} = \frac{\exp(\lambda_m IV_{nm})}{\sum_{m'} \exp(\lambda_{m'} IV_{nm'})}. \quad (9)$$

重点检查 nest parameter 的自然参数约束：

$$0 < \lambda_m \leq 1. \quad (10)$$

如果你的库内部用 unconstrained transform, 例如

$$\lambda_m = \sigma(\rho_m), \quad (11)$$

那么输出和对比时必须转换回 Apollo 报告的自然参数。

7.3 Step 3: Mixed logit

Mixed logit 是最容易出现差异的模型。必须控制：

- same data;

- same initial values;
- same random seed;
- same number of draws;
- same draw type;
- same distribution;
- same panel grouping;
- same fixed/random parameters;
- same optimizer tolerance。

Mixed logit 的比较分两步。

第一步：固定 θ_0 下比较 likelihood 给定同一组参数 θ_0 ，比较：

$$\ell_{\text{Apollo}}(\theta_0) \quad \text{vs.} \quad \ell_{\text{torch}}(\theta_0). \quad (12)$$

如果这一步不一致，说明 likelihood、变量 scaling、availability、panel grouping 或 draws 没有对齐。

第二步：比较最终估计值 确认 likelihood 一致之后，再比较：

$$\hat{\theta}_{\text{Apollo}} \quad \text{vs.} \quad \hat{\theta}_{\text{torch}}. \quad (13)$$

Panel mixed logit 必须使用 individual-level likelihood：

$$L_n(\theta) = \int \prod_{t=1}^{T_n} P_{nty_{nt}}(\beta) f(\beta | \theta) d\beta. \quad (14)$$

不能错误地写成每个 choice occasion 独立积分：

$$\prod_{t=1}^{T_n} \int P_{nty_{nt}}(\beta) f(\beta | \theta) d\beta. \quad (15)$$

7.4 Step 4: Latent class

Latent class probability:

$$P_{ni} = \sum_{q=1}^Q \pi_{nq} P_{ni}^{(q)}. \quad (16)$$

Class allocation model:

$$\pi_{nq} = \frac{\exp(Z_n \gamma_q)}{\sum_{q'} \exp(Z_n \gamma_{q'})}. \quad (17)$$

比较项目：

- class-specific parameters;
- class membership parameters;

- posterior class probabilities;
- log-likelihood;
- market shares by class。

注意 label switching 问题。比较时应允许 class index 置换：

$$\min_{\pi \in \Pi_Q} \sum_q \left\| \hat{\theta}_q^{\text{torch}} - \hat{\theta}_{\pi(q)}^{\text{Apollo}} \right\|. \quad (18)$$

7.5 Step 5: Hybrid choice / ICLV

ICLV 最后再做。建议先用 synthetic data, 因为真实数据中 latent variable 没有真值, debug 很难。

联合 likelihood:

$$L_n = \int P(y_n | X_n, x_n^*; \theta) \prod_m P(I_{nm} | x_n^*; \alpha_m) f(x_n^* | Z_n; \gamma) dx_n^*. \quad (19)$$

8 Python 端比较脚本

8.1 结果读取和参数对比

```
import json
import numpy as np
import pandas as pd

def load_json(path):
    with open(path, "r") as f:
        return json.load(f)

apollo = load_json("benchmarks/results/apollo_mnl_swissmetro.json")
torch = load_json("benchmarks/results/torchdcm_mnl_swissmetro.json")

rows = []
for name, a_hat in apollo["estimates"].items():
    t_hat = torch["estimates"][name]
    abs_diff = abs(a_hat - t_hat)
    rel_diff = abs_diff / max(1e-12, abs(a_hat))
    rows.append({
        "parameter": name,
        "apollo": a_hat,
        "torchdcm": t_hat,
        "abs_diff": abs_diff,
        "rel_diff": rel_diff,
    })

df = pd.DataFrame(rows)
print(df)

print("LL Apollo:", apollo["loglik"])
```

```
print("LL TorchDCM:", torch["loglik"])
print("LL diff:", abs(apollo["loglik"] - torch["loglik"]))
```

8.2 自动化断言

MNL/NL 等 deterministic likelihood 模型可以设置较严格 tolerance:

```
assert abs(apollo["loglik"] - torch["loglik"]) < 1e-5
assert df["abs_diff"].max() < 1e-4
assert probability_diff.max() < 1e-6
```

Mixed logit 的 tolerance 应更谨慎。除非 Apollo 和 TorchDCM 使用完全相同 draws, 否则最终估计值不应设置过严。

9 Benchmark 数据集

数据集	用途	优先级
Apollo built-in example data	直接和 Apollo 官方例子对齐。	高
apollo_modeChoiceData	simulated mode choice data, 含 500 individuals、8,000 choices、availability、alternative attributes 和 individual characteristics。	高
Swissmetro	交通 DCM 经典数据, 适合和 Biogeme/Apollo/文献对齐。	高
Synthetic MNL data	知道真值, 用于验证 consistency、bias、SE coverage。	高
Large synthetic data	测试 CPU/GPU speed、memory、ragged choice-set kernel。	高
Panel synthetic MMNL	测试 panel mixed logit、cluster SE、posterior individual parameters。	高
Latent class synthetic data	测试 EM、label switching、class posterior。	中
ICLV synthetic data	测试 latent variable 和 measurement equation。	中

10 常见坑与处理原则

10.1 变量 scaling

Apollo 例子里经常会对 time、cost 做缩放, 例如 minutes 除以 60, cost 除以 100。TorchDCM 必须完全一致, 否则参数会差一个数量级。

10.2 ASC normalization

MNL 需要固定一个 alternative-specific constant, 例如:

$$ASC_{SM} = 0. \quad (20)$$

Apollo 和 TorchDCM 必须固定同一个 base alternative。

10.3 Availability

某个 alternative 不可用时, 不能只是把 utility 设为 0, 而必须从 choice set 中移除, 或者等价地设为 $-\infty$ 。这是很多实现误差的来源。

10.4 Panel likelihood

Repeated SP/RP 数据在 mixed logit 中不能简单当成 independent observations。必须按 individual 做 panel likelihood。

10.5 Random draws

Mixed logit 如果 draws 不一样, 估计值可能略有差异。建议:

1. 先比较 deterministic models, 如 MNL/NL;
2. mixed logit 先比较 fixed-theta simulated likelihood;
3. 尽量使用 common random numbers;
4. 报告 draw type、draw count、seed;
5. 若无法保证同 draws, 增加 draws 并放宽 tolerance。

10.6 Optimizer convergence

Apollo 和 PyTorch 的 optimizer 不同, 最终可能到达数值上略有差异的点。建议分成两个测试:

- Test A: same θ , 比较 likelihood/probability, 用于 implementation correctness;
- Test B: independent estimation, 比较 optimum, 用于 estimation performance。

11 可交付 benchmark 工具

你的库可以内置一个 optional benchmark 工具:

```
from torchdcm.benchmarks import ApolloBenchmark

bench = ApolloBenchmark(
    model="mnl",
    data="swissmetro",
    spec="benchmarks/specs/swissmetro_mnl.json",
    apollo_script="benchmarks/apollo/R/run_mnl.R",
)

report = bench.run()
report.summary()
```

```
report.to_markdown("benchmarks/results/report.md")
report.to_latex("benchmarks/results/report.tex")
```

命令行接口:

```
torchdcm-benchmark --backend apollo --model mnl --data swissmetro
torchdcm-benchmark --backend apollo --model nl --data swissmetro
torchdcm-benchmark --backend apollo --model mixed_logit --data
    apollo_mode_choice
```

12 开发里程碑

阶段	任务	完成标准
M0	搭建 benchmark 目录、JSON spec、Apollo Rscript runner。	Python 能调用 Apollo 并读取 JSON。
M1	MNL 对齐。	参数、LL、SE、概率与 Apollo 一致。
M2	NL/CNL 对齐。	nest 参数、inclusive value、概率一致。
M3	Inference 对齐。	classic/robust/cluster SE、WTP、elasticity 可复现。
M4	Mixed logit 对齐。	fixed-theta likelihood 一致，估计值接近。
M5	Runtime/memory benchmark。	输出标准 benchmark report。
M6	Latent class。	处理 label switching，并输出 posterior class probability。
M7	ICLV synthetic benchmark。	joint likelihood 和 latent/measurement parameters 对齐。

13 论文或文档中的 benchmark 描述

可以在软件论文或技术文档中这样描述:

We use Apollo as an external reference implementation rather than a Python dependency. For each benchmark, we export a shared model specification, dataset, initial parameter vector, fixed-parameter list, random seed, and draw configuration. Apollo is executed through Rscript, and its estimates, standard errors, log-likelihood, and post-estimation outputs are exported to JSON. Our PyTorch implementation reads the same specification and data, estimates the same model, and compares parameter estimates, likelihood values, predicted

probabilities, covariance matrices, post-estimation quantities, runtime, and memory usage.

中文表述可以写成：

我们将 Apollo 作为外部 reference implementation，而不是 Python 依赖。每个 benchmark 使用同一份数据、模型规格、初始参数、固定参数、随机种子和 draws 设置。Apollo 通过 Rscript 运行，输出 JSON 格式的估计值、标准误、log-likelihood 和 post-estimation 结果。PyTorch 实现读取同样的规格和数据，估计同一模型，并比较参数、似然值、预测概率、协方差矩阵、post-estimation 指标、运行时间和内存使用。

14 最小可执行计划

第一个可执行任务建议定为：

Benchmark 1: Apollo MNL vs TorchDCM MNL on apollo_modeChoiceData.

具体目标：

- same utility specification;
- same fixed ASC;
- same initial values;
- same availability;
- same variable scaling;
- same log-likelihood;
- same parameters within 10^{-4} ;
- same predicted probabilities within 10^{-6} ;
- same classical standard errors within 10^{-4} .

第二个任务：

Benchmark 2: Apollo MMNL vs TorchDCM MixedLogit.

具体目标：

- same data;
- same number of draws;
- same draw type and seed;
- same random coefficient distribution;
- same panel likelihood;
- compare fixed-theta simulated likelihood first;
- then compare final estimates and posterior individual parameters.

15 参考来源

- Apollo official site: <https://www.apollochoicemodelling.com/>
- CRAN Apollo package page: <https://cran.r-project.org/package=apollo>
- Apollo examples and data: <https://www.apollochoicemodelling.com/examples.html>
- Apollo modeChoiceData documentation: https://rdr.io/cran/apollo/man/apollo_modeChoiceData.html
- rpy2 documentation: <https://rpy2.github.io/>
- scikit-learn estimator development guide: <https://scikit-learn.org/stable/developers/develop.html>
- statsmodels discrete models documentation: <https://www.statsmodels.org/stable/discretemod.html>